
ASSIGNMENTS

Problem set 6

AI504 — Knowledge Representation

Simon Holm · Johannes Rothe · Shuagib Ibrahim · Anne Sofie Høj · Daniel Nissen

May, 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1 Problem 1	3
2 Problem 2	7
3 Problem 3	11

Problem 1

We are going to describe a translation from the syntax of $\mathcal{A}(\mathcal{RC})$ to the syntax of propositional logic. First of all, if (P, R) is a signature of $\mathcal{A}(\mathcal{RC})$ then just map it to the propositional logic signature $P \cup R$. (In other words, either a noun or a verb in my source signature can be used as a sentence letter in my target signature.)

Now map terms t of $\mathcal{A}(\mathcal{RC})$ to formulas t^* of propositional logic as follows. Each noun p becomes the propositional formula p . Then “ r all (...)” becomes “ $(...) \rightarrow r$,” so that, e.g., the term `see love all dogs` becomes $(\text{dogs} \rightarrow \text{love}) \rightarrow \text{see}$. In the latter formula `see`, `love`, `dogs` are just boolean-valued sentence letters; we have forgotten the noun/verb distinction.

Finally, the translation of the sentence “ t all s ” is $t^* \rightarrow s^*$, where t^* and s^* is the mapping on terms defined above. (Call this map $\phi \mapsto \phi^*$, reusing the symbol $*$.) So, for example

$$(\text{all} (\text{see all love all dogs}) (\text{love all birds}))^*$$

- (a) Suppose that Γ is a set of sentences and ϕ is a sentence in $\mathcal{A}(\mathcal{RC})$, and suppose that $\Gamma \models \phi$. Prove that $\Gamma^* \models \phi^*$, as a formulas of propositional logic.

Solution:

This is proved by induction in an $\mathcal{A}(\mathcal{RC})'$.

Proof by induction

Goal: For all T, Γ, ϕ where T is a proof tree s.t. $\Gamma \vdash \phi$, then there exists a proof tree T^* s.t. $\Gamma^* \vdash \phi^*$.

1 Base case

2 If T is a single sentence (the whole tree is a leaf), then either ϕ is an axiom such that $\phi = \phi^*$ or $\phi \in \Gamma$ which means that $\phi^* \in \Gamma^*$

3 Then since all leaves are $\in \Gamma^*$, $\Gamma^* \vdash \phi^*$ and from soundness of propositional logic, $\Gamma^* \models \phi^*$

4 Inductive hypothesis

5 For all proofs T where $\Gamma \vdash \phi$, there exists another proof T^* from Γ^* such that $\Gamma^* \vdash \phi^*$

6 Inductive step

7 Suppose T is not a leaf, then it is either an instance of ANTI or BARBARA.

8 Case 1 ANTI:

9 If T is not a single leaf but an instance of ANTI, then it is a sentence proved by one subtree $\mathcal{T}$. This can be written up as follows:

$$\frac{\frac{\overbrace{\text{all } x \ y}^{\mathcal{T}}}{\text{all } (r \text{ all } y) \ (r \text{ all } x)}}{\phi} \text{ANTI}$$

11 Applying the conversion rules for $\mathcal{A}(\mathcal{RC})$ to propositional logic yields the converted subtree $\mathcal{T}^*$ and sentence ϕ^* :

12

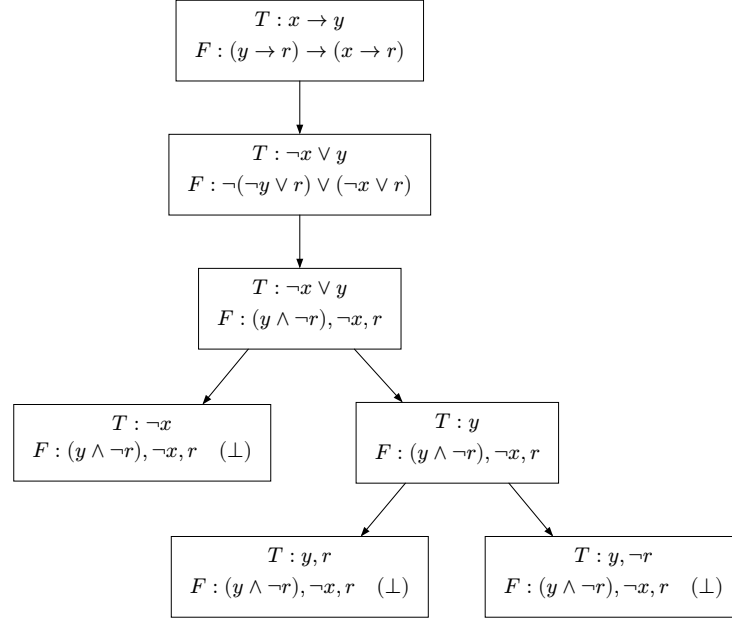
$$\frac{\overbrace{x \rightarrow y}^{\mathcal{T}^*}}{\underbrace{(y \rightarrow r) \rightarrow (x \rightarrow r)}_{\phi^*}} \text{ANTI}$$

13

Then to show that $x \rightarrow y \implies (y \rightarrow r) \rightarrow (x \rightarrow r)$.

This can be done by assuming that $(y \rightarrow r) \rightarrow (x \rightarrow r)$ is false and exploring all nodes in an analytic tableau-style table to ensure that all ends are contradictions:

14



15

Since all leaves lead to a contradiction denoted as $(\perp)$. This proves ϕ^*

16

By the basis of the inductive hypothesis applied to $\mathcal{T}$, it follows that $\Gamma^* \vdash \mathcal{T}^*$, and by using the ANTI rule, it follows that $\Gamma^* \vdash \phi^*$.

17

Case 2: BARBARA

18

If T is not a single leaf, but an instance of BARBARA, then it is a sentence proved by 2 subtrees $\mathcal{T}_0, \mathcal{T}_1$ which can be written as follows:

19

$$\frac{\overbrace{(\text{all } p \ x)}^{\mathcal{T}_0} \quad \overbrace{(\text{all } x \ q)}^{\mathcal{T}_1}}{\underbrace{\text{all } p \ q}_{\phi}} \text{BARBARA}$$

20

Where T^* would be:

21

Applying the conversion rules for $\mathcal{A}(\text{RC})$ to propositional logic yields the converted subtrees $\mathcal{T}_0^*, \mathcal{T}_1^*$ and sentence ϕ^* :

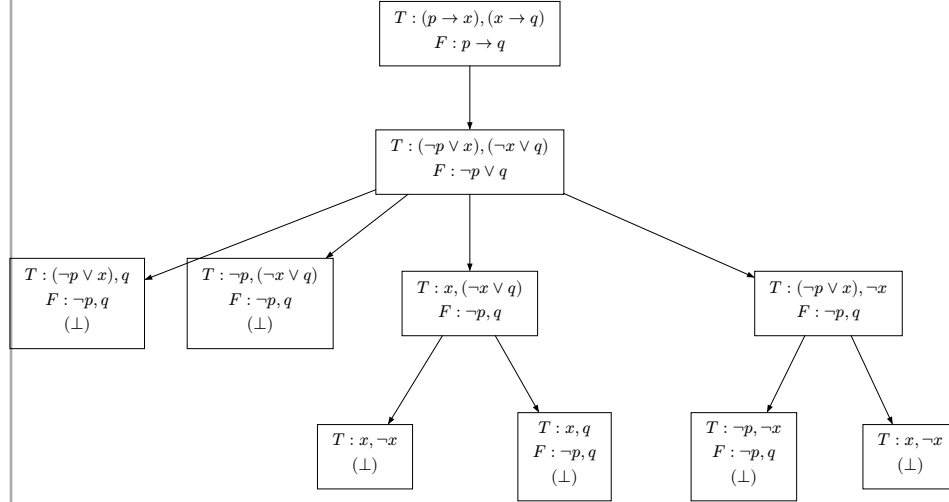
22

$$\frac{\overbrace{(p \rightarrow x)}^{\mathcal{T}_0^*} \quad \overbrace{(x \rightarrow q)}^{\mathcal{T}_1^*}}{\underbrace{p \rightarrow q}_{\phi^*}} \text{BARBARA}$$

23

Using the same tableau method as before, it can be shown that the two subtrees prove ϕ^* :

24



25

Therefore, all proofs T from $\mathcal{A}(\mathcal{RC})'$ where $\Gamma \vdash \phi$, proves the existence of $\Gamma^* \vdash \phi^*$, and from propositional logic soundness, $\Gamma^* \models \phi^*$

- (b) Show that the converse is not true. In other words, come up with a concrete sentences Γ, ϕ of $\mathcal{A}(\mathcal{RC})$ such that $\Gamma^* \models \phi^*$ in propositional logic but $\Gamma \not\models \phi$ in $\mathcal{A}(\mathcal{RC})$.

Take the sentence $\phi = \text{all } x (z \text{ all } y)$ and $\Gamma = \{\text{all } y (z \text{ all } x)\}$. To show that $\Gamma \not\models \phi$, a concrete model will be used as a counterexample. Let:

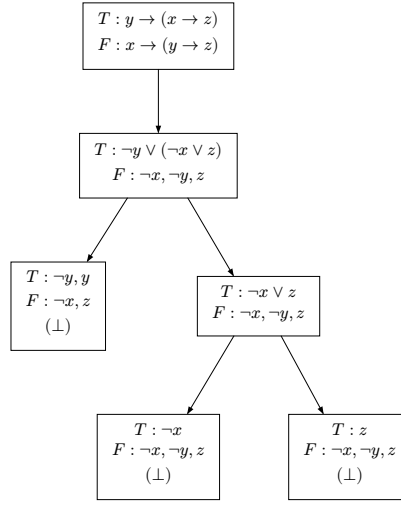
$$\begin{aligned}\llbracket x \rrbracket &= \{1\} \\ \llbracket y \rrbracket &= \{2\} \\ \llbracket z \rrbracket &= \{(2, 1)\}\end{aligned}$$

Then:

$$\begin{aligned}\llbracket z \text{ all } x \rrbracket &= \{2\} \\ \llbracket z \text{ all } y \rrbracket &= \emptyset\end{aligned}$$

Since $\{1\} \notin \emptyset$, this counter model shows that $\Gamma \not\models \phi$.

By the rules stated in Section 1, ϕ becomes $\phi^* = x \rightarrow (y \rightarrow z)$ and $\Gamma^* = \{y \rightarrow (x \rightarrow z)\}$. Now lets use tableau-style proof, to show that $\Gamma^* \vdash \phi^*$.



This shows that in all possible models of Γ^* , $\Gamma^* \models \phi^*$

Problem 2

If ϕ is a sentence in propositional logic, let $\phi^\dagger$ be obtained from ϕ by negating all of the sentence letters. For example,

$$((p \vee (q \implies r))^\dagger \iff \neg(r \wedge p))^\dagger = (\neg p \vee (\neg q \implies \neg r)) \iff \neg(\neg r \wedge \neg p)$$

- (a) For any satisfiable propositional formula ϕ , must $\phi^\dagger$ also be satisfiable? Either prove or give a counterexample.

Solution:

The goal is to show that for any satisfiable propositional formula ϕ , $\phi^\dagger$ is also satisfiable.

ϕ is satisfiable if there is some function v such that finds a truth assignment such that the sentence evaluates to true:

$$\exists v. P \rightarrow Bool : \llbracket \phi \rrbracket_v = T$$

$\phi^\dagger$ is obtained from ϕ by negating all of the sentence letter. For it to be satisfiable, there must also be a function v' s.t.

$$\exists v'. P \rightarrow Bool : \llbracket \phi^\dagger \rrbracket_{v'} = T$$

v' being a function that assigns the opposite truth value to each sentence letter from v :

$$v'(x) = \begin{cases} F & v(x) = T \\ T & v(x) = F \end{cases}$$

Since $\phi^\dagger$ is obtained by negating every sentence letter in ϕ and v' negates the truth value of each sentence letter. Evaluating $\phi^\dagger$ with v' is the same as negating each letter and flipping it assigned truth value. This can be shown using induction:

Proof by induction

Goal: Show that $\llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v$ for all sentences ϕ

Base case

Case 1

Let ϕ be an atomic sentence p

Then $\llbracket \neg p_i \rrbracket_{v'} = \llbracket p_i \rrbracket_v$,

$\neg \llbracket p_i \rrbracket_{v'} = \llbracket p_i \rrbracket_v$

And so $\neg v'(p_i) = v(p_i)$ is trivial by v' definition

Case 2

Let ϕ be T or F :

Then $\llbracket T^\dagger \rrbracket_{v'} = \llbracket T \rrbracket_v$

$\llbracket \neg T \rrbracket_{v'} = \llbracket T \rrbracket_v$

$\neg \llbracket T \rrbracket_{v'} = \llbracket T \rrbracket_v$

$\neg v'(T) = v(T)$ which is trivial by v' definition, and holds for F aswell.

Inductive hypothesis

Assume that $S(\phi) \stackrel{\text{def}}{=} \llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v$

Inductive step

ϕ can either be negated ($\neg$) or used with any other operator ($\vee, \wedge, \implies$ or $\iff$)

Case 1 (negation):

Show that $S(\phi) \implies S(\neg\phi)$

By **IH**, this is:

$$\llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \llbracket \neg\phi^\dagger \rrbracket_{v'} = \llbracket \neg\phi \rrbracket_v$$

And thus:

$$\llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \neg \llbracket \phi^\dagger \rrbracket_{v'} = \neg \llbracket \phi \rrbracket_v$$

$$\llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \neg \llbracket \phi^\dagger \rrbracket_{v'} = \neg \llbracket \phi \rrbracket_v$$

$$\llbracket \neg\phi \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \neg \llbracket \neg\phi \rrbracket_{v'} = \neg \llbracket \phi \rrbracket_v$$

$$\neg \llbracket \phi \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \llbracket \phi \rrbracket_{v'} = \neg \llbracket \phi \rrbracket_v$$

$$\neg \llbracket \phi \rrbracket_{v'} = \llbracket \phi \rrbracket_v \implies \llbracket \phi \rrbracket_{v'} = \neg \llbracket \phi \rrbracket_v$$

$$\neg v'(\phi) = v(\phi) \implies v'(\phi) = \neg v(\phi)$$

Which is trivially true by v' definition

Case 2 (other operators)

Let $\odot$ be any of the following logical operators $\wedge, \vee, \implies$ or $\iff$.

Show that if $S(\phi) \odot S(\psi) \implies S(\phi \odot \psi)$

Then

$$\begin{array}{l}
\left(\llbracket \phi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v \right) \odot \left(\llbracket \psi^\dagger \rrbracket_{v'} = \llbracket \psi \rrbracket_v \right) \implies \llbracket (\phi \odot \psi)^\dagger \rrbracket_{v'} = \llbracket \phi \odot \psi \rrbracket_{v'} \\
\implies \llbracket \phi^\dagger \rrbracket_{v'} \odot \llbracket \psi^\dagger \rrbracket_{v'} = \llbracket \phi \rrbracket_v \odot \llbracket \psi \rrbracket_v \\
(\llbracket \neg \phi \rrbracket_{v'} = \llbracket \phi \rrbracket_v) \odot (\llbracket \neg \psi \rrbracket_{v'} = \llbracket \psi \rrbracket_v) \implies \llbracket \neg \phi \rrbracket_{v'} \odot \llbracket \neg \psi \rrbracket_{v'} = \llbracket \phi \rrbracket_v \odot \llbracket \psi \rrbracket_v \\
(\neg \llbracket \phi \rrbracket_{v'} = \llbracket \phi \rrbracket_v) \odot (\neg \llbracket \psi \rrbracket_{v'} = \llbracket \psi \rrbracket_v) \implies \neg \llbracket \phi \rrbracket_{v'} \odot \neg \llbracket \psi \rrbracket_{v'} = \llbracket \phi \rrbracket_v \odot \llbracket \psi \rrbracket_v \\
(\neg v'(\phi) = v(\phi)) \odot (\neg v'(\psi) = v(\psi)) \implies \neg v'(\phi) \odot \neg v'(\psi) = v(\phi) \odot v(\psi)
\end{array}$$

27 This is trivial by definition of v' and applies to all logical operators.

The proof by induction shows that:

$$\llbracket [\phi^\dagger] \rrbracket_{v'} = \llbracket [\phi] \rrbracket_v$$

And from the definition of satisfiability:

$$\exists v : \llbracket [\phi] \rrbracket_v = T \implies \exists v' : \llbracket [\phi^\dagger] \rrbracket_{v'} = T$$

- (b) Find a formula ϕ such that ϕ is neither a tautology nor a contradiction and ϕ and $\phi^\dagger$ are logically equivalent? Find one or prove that none exists.

Assume that ϕ is satisfiable and falsifiable which means:

$$\begin{array}{l}
\exists v. P \rightarrow \text{Bool} : \llbracket [\phi] \rrbracket_v = T \\
\exists f. P \rightarrow \text{Bool} : \llbracket [\phi] \rrbracket_f = F
\end{array}$$

From definition 5.8, two sentences ϕ, ψ are logically equivalent if:

$$\forall v : \llbracket \phi \rrbracket_v = \llbracket \psi \rrbracket_v$$

Let's now assign

$$\begin{array}{l}
\phi := p \iff q \\
\phi^\dagger := \neg p \iff \neg q
\end{array}$$

and from the formulas truth table of possible models:

p	q	$\neg p$	$\neg q$	$p \iff q$	$\neg p \iff \neg q$
T	T	F	F	T	T
T	F	F	T	F	F
F	T	T	F	F	F
F	F	T	T	T	T

Table 1: truth table of ϕ and $\phi^\dagger$,
showing that both ϕ and $\phi^\dagger$ is satisfiable

From Table 1 it can be seen that for both sentences, there exists both a v, f such that the models evaluate to T and F . For all possible models of assignment for each sentence letter, both sentences evaluates to the same, therefore $\phi \equiv \phi^\dagger$.

Problem 3

Work in the signature that has sentence letters $p_{i,j}$ for $0 \leq i, j \leq 9$ (one hundred letters in all). Consider the formula Φ which is the conjunction of all of the following clauses:

- $p_{0,9} \wedge p_{9,0}$,
- $p_{i,j} \implies p_{i-1,j}$ for each $1 \leq i \leq 9$ and $0 \leq j \leq 9$, and
- $p_{i,j} \implies p_{i,j-1}$ for each $0 \leq i \leq 9$ and $1 \leq j \leq 9$.

How many models satisfy Φ ? (*Hint.* If this seems too daunting, replace 9 with something smaller.)

Solution

First of all to complete this problem it is necessary to figure out what the first possible state of the space is based on the given clauses. It is known from the clauses that both $p_{0,9}$ and $p_{9,0}$ need to be filled out, as this is the first given clause, the next two clauses then also state something that based on the two original points will create a change in the starting space. The 2nd clause states that every cell to the left of a filled out cell, will also need to be filled out, and the third clause states that every cell below any colored cell will also need to be colored.

Then the first state of all the cells can be visualized in a table form like this, where the red cells are the ones filled by the first clause:

x									
x									
x									
x									
x									
x									
x									
x									
x									
x	x	x	x	x	x	x	x	x	x

Then the next step would be to figure out how much is needed to actually fill all the cells, when looking at the build of the table and the clauses, we can sort of imagine that some steps, either normal, steep or very wide would happen when attempting to fill out in every different combination of cells. And since it is a lot of combinations, the logical way forwards would be using combinatorics.

We can see that the total amount of cells from all the way left to right or top to bottom is 10, so essentially the grid is 10×10 but since the first row and column is filled by the clauses we know that it is know a 9×9 grid, and due to the 2nd and 3rd clause we know that they will fill out in a way that fills a lot of cells and combinations at a time, meaning that we can essentially only take 9 steps to the right or 9 steps up.

Now it can just be entered into the combination formula given on form:

$${}^nC_r = \frac{n!}{r! \cdot (n-r)!}$$

Where n is the total amount of moves that can be done, and r is the max amount of moves that can be done in one step.

Then we can add the numbers from our actual task where we know that there is essentially a total of 18 steps and 9 steps that can be taken in one step.

$${}^{18}C_9 = \frac{18!}{9! \cdot (18-9)!} = 48620$$

Meaning that there is a total of 48620 models that satisfy the given clauses.